

Documentation Generator — Submission

This document walks through the design, build, review, and automation of a documentation generator for this repository (a React + Vite personal site). It covers: what the generator does, the project's CLAUDE.md, the first-version output and a critical review of it, the Plan Mode plan used for the v2 rewrite, the custom slash command and hook that automate it, the permissions guardrails put around that automation, and one full end-to-end automated run.

(a) Short description of the generator

`scripts/generate-docs.js` (with logic split across `scripts/docgen/parse.js`, `graph.js`, and `render.js`) reads every `.js / .jsx` file under `src/` and writes markdown documentation into `docs/`: one file per source file plus a `docs/README.md` index. It is regex-based, not an AST parser — deliberately kept dependency-free and simple for this codebase's plain function-component style.

Per file, it reports: - **Description** — from a leading `/** ... */` block, or a fallback derived from the JSX root element's `id` attribute. - **Imports** and, via a reachability analysis from `src/main.jsx`, an "Imported by" list. - **Exports vs. internal helpers** — components split into what the file actually exposes vs. private, non-exported helpers. - **Local data constants** — top-level `UPPER_CASE` arrays/objects. - **Orphan detection** — a `⚠ Not imported anywhere` banner (and index-table marker) for any file unreachable from the app's entry point, so dead code is flagged automatically instead of documented with false authority.

It runs via `npm run docs`, a `/generate-docs` slash command, and automatically after any edit under `src/` via a `PostToolUse` hook (see sections e–g).

(b) CLAUDE.md

```
# CLAUDE.md
```

```
This file provides guidance to Claude Code (claude.ai/code) when working with code in t
```

```
## Commands
```

- `npm run dev` – start the Vite dev server
- `npm run build` – production build to `dist/`
- `npm run preview` – preview the production build locally
- `npm run docs` – regenerate `docs/` from `src/` (see Documentation below)

No lint or test setup exists in this repo. `npm run build` succeeding is the only correct

Folder structure

index.html Vite entry point src/ main.jsx ReactDOM root render App.jsx Tab shell (About / Research / Projects / CV), local useState, no router index.css Tailwind directives + custom @layer utility classes components/ one component per file public/ static passthrough dir → copied to dist/ root; referenced by absolute path from components (/img/*, /BUET-Curriculum.html, /utd-Curriculum.html) docs/ generated documentation — do not hand-edit, see Documentation below scripts/generate-docs.js orchestrator: walks src/, calls docgen/, writes docs/ scripts/docgen/ parse.js (extraction), graph.js (import-reachability), render.js (markdown) assets/, forms/, dead legacy Bootstrap "Kelly" CV template — not part of the Vite build, BUET-Curriculum.html (root) not referenced by src/ or index.html. Don't extend; prefer removing. dist/ gitignored local build artifact, not tracked in git

Conventions

- Function components only (`export default function Name() {}`); no class components,
- Page sections are tabs held in `App.jsx`'s local `useState`, not routes.
- No CMS/API/markdown: page content is hardcoded JS arrays/objects inside the component
- Styling is Tailwind utility classes only — no CSS modules, no styled-components. Custom
- `src/components/Hero.jsx`, `Research.jsx`, `News.jsx`, `Skills.jsx` are orphaned — no

Documentation

`docs/` is generated by `npm run docs` (`scripts/generate-docs.js` + `scripts/docgen/`)

Per file it reports: description (from a leading `/\*\* ... \*/` block, else a fallback ba

The generator is regex-based (`scripts/docgen/parse.js`), not an AST parser, so write s

Deployment

`.github/workflows/deploy.yml` builds and deploys on every push to `main` (or manual `v

(c) First-version output and critique

v1 output samples

The first version (a single monolithic `scripts/generate-docs.js`, no orphan detection, no exported/helper distinction) produced output like this.

`docs/README.md` (v1):

```
# Source Documentation

Auto-generated by `npm run docs` (`scripts/generate-docs.js`). Do not edit by hand – re-run the script.

| File | Components | Description |
|---|---|---|
| [src/App.jsx](./src/App.jsx.md) | App | – |
| [src/components/About.jsx](./src/components/About.jsx.md) | About, SectionTitle | – |
| [src/components/CV.jsx](./src/components/CV.jsx.md) | CV, TimelineItem | – |
| [src/components/Footer.jsx](./src/components/Footer.jsx.md) | Footer | – |
| [src/components/Hero.jsx](./src/components/Hero.jsx.md) | Hero | – |
| [src/components/Navbar.jsx](./src/components/Navbar.jsx.md) | Navbar | – |
| [src/components/News.jsx](./src/components/News.jsx.md) | News | – |
| [src/components/Projects.jsx](./src/components/Projects.jsx.md) | Projects | – |
| [src/components/Publications.jsx](./src/components/Publications.jsx.md) | Publication | – |
| [src/components/Research.jsx](./src/components/Research.jsx.md) | Research | – |
| [src/components/SectionHeader.jsx](./src/components/SectionHeader.jsx.md) | SectionHeader | – |
| [src/components/Sidebar.jsx](./src/components/Sidebar.jsx.md) | Sidebar | – |
| [src/components/Skills.jsx](./src/components/Skills.jsx.md) | Skills | – |
| [src/main.jsx](./src/main.jsx.md) | – | – |
```

`docs/src/components/CV.jsx.md` (v1):

```
# CV.jsx

`src/components/CV.jsx`

## Imports
- `SectionHeader` from `./SectionHeader`

## Components / Functions
### `CV()`
No props.

### `TimelineItem(title, subtitle, period, institution, children, link, linkLabel)`
Props: `title`, `subtitle`, `period`, `institution`, `children`, `link`, `linkLabel`
```

docs/src/components/SectionHeader.jsx.md (v1):

```
# SectionHeader.jsx

`src/components/SectionHeader.jsx`

## Imports
_None_

## Components / Functions
### `SectionHeader(title, subtitle)`
Props: `title`, `subtitle`
```

Critique

What's accurate: imports, component names, and prop lists are all factually correct. Every destructured prop list matches the source exactly. Data-constant detection works well (SOCIALS, PROJECTS, TAG_COLORS, etc. all correctly picked up). Zero-component and zero-import files render sensibly rather than breaking.

Where it's vague: every "Description" is — (no JSDoc exists anywhere in the codebase, so the generator has nothing to summarize). Signature format is misleading: `SectionHeader(title, subtitle)` reads like positional args, but the real signature is `SectionHeader({ title, subtitle })` — destructured object props. "Local data constants" lists bare names with no shape.

Confident hallucinations (the important finding): the generator presents every file with identical authority regardless of whether it's actually live code. `Research.jsx` and `Publications.jsx` read almost identically in the v1 docs, but only `Publications` is wired into `App.jsx`; `Research.jsx`, `Hero.jsx`, `News.jsx`, `Skills.jsx` are never imported anywhere — and nothing in the v1 output says so. A reader relying only on `docs/` would reasonably conclude all 14 files ship. Similarly, `CV.jsx`'s index row lists `CV`, `TimelineItem` as if both were part of the module's public surface, but `TimelineItem` is an unexported local helper. These aren't fabricated facts, but they create a false impression of uniformity and completeness.

Improvements identified (priority order): 1. Cross-reference the import graph from `src/main.jsx` and mark unreachable files/components — highest value, since the orphan-file problem is real and was invisible. 2. Mark exported vs. non-exported symbols separately. 3. Fix signature rendering to show `{ title, subtitle }` when destructured, not positional args. 4. Support arrow-function local helpers (`const Foo = () => {}`) — only `function Foo(...)` declarations were caught. 5. Infer a fallback description from the JSX root element's

`id` /section role when no JSDoc exists. 6. Surface the calling context (which parent renders this) — implemented generically as the graph's reverse "Imported by" edges rather than a brittle, App.jsx-specific heuristic.

All six items were carried into the v2 rewrite (see Plan Mode plan below); items 1–5 were implemented, item 6 was generalized as described.

(d) Plan Mode plan

Written via `EnterPlanMode` → `ExitPlanMode`, approved by the user before any implementation:

```
# Plan: Multi-file improvement to the docs generator
```

```
## Context
```

```
`scripts/generate-docs.js` (v1) was built and reviewed earlier this session. The criticisms
```

- **Confident hallucination by omission**: dead/orphaned components (``Hero.jsx``, ``Research.jsx``)
- Exported symbols and unexported local helpers (e.g. ``CV.jsx``'s ``TimelineItem``) are lost
- Signatures render destructured props as positional args (``SectionHeader(title, subtitle)``)
- Arrow-function local helpers (``const Foo = () => {}``) are silently invisible to the crawler
- No file gets a description unless it has a top-of-file JSDoc block (none currently exist)

```
This plan fixes the orphan-detection gap (highest value) plus the three cheap, mechanical
```

```
## File layout
```

```
Split `scripts/generate-docs.js` into:
```

- **``scripts/docgen/parse.js``** — per-file extraction (moved from the current script, ``scripts/generate-docs.js``)
 - ``extractComponents`` gains a third regex for non-exported arrow-function helpers (``const``)
 - ``parseProps`` returns ``{ names, destructured }`` instead of a bare array, so rendering is easier
 - New ``extractSectionId(source)`` — cheap regex for the first ``id="..."`` literal in the file
 - ``extractFileDescription``, ``extractImports``, ``extractDataConstants`` move here unchanged
- **``scripts/docgen/graph.js``** (new) — given the full file list and each file's parsed data, build the graph
- **``scripts/docgen/render.js``** (new) — takes one file's parsed data + its graph info, renders it
- **``scripts/generate-docs.js``** — thin orchestrator: walk ``src/``, call ``parse`` per file, then ``graph``, then ``render``

```
No new dependencies; still ESM (`type: module` already set in `package.json`).
```

```
## What's explicitly out of scope for this pass
```

- No AST parser (babel/acorn) – regex stays the good-enough tool for this codebase's purposes
- No hardcoded "rendered when activeTab === X" narrative parsing (too brittle/one-off for future)
- No test framework addition – repo has none today; verification is manual (see below)

Verification

1. ``npm run docs`` – regenerate ``docs/``.
2. Read ``docs/src/components/Research.jsx.md`` (or ``Hero.jsx``/``News.jsx``/``Skills.jsx``) and confirm ``TimelineItem`` is listed under "Interactions".
3. Read ``docs/src/components/CV.jsx.md`` and confirm ``TimelineItem`` is listed under "Interactions".
4. Confirm ``docs/README.md``'s table has an orphan indicator and that it's blank/absent.
5. ``npm run build`` – confirm the unrelated Vite build still passes (sanity check nothing broke).
6. Update the "Documentation" section of ``CLAUDE.md`` to mention orphan-detection, export

All six verification steps were run after implementation and passed (see section g for one live example).

(e) Custom slash command and hook configuration

Slash command — `.claude/commands/generate-docs.md`

```
---
description: Regenerate docs/ from src/ using the project's documentation generator
---
```

Run this project's documentation workflow in one step:

1. Run ``npm run docs`` from the repository root. This executes ``scripts/generate-docs.js``.
2. Report how many files were generated (from the script's console output).
3. Read ``docs/README.md`` and tell the user which files, if any, are flagged ``△`` (orphaned).

Do not hand-edit anything under ``docs/`` – it is entirely regenerated by this command.

Invoked as `/generate-docs` — the manual trigger.

Hook — `.claude/settings.json`

```
{
  "permissions": {
    "deny": [
      "Bash(rm -rf *)",
      "Bash(git push --force*)",
      "Bash(git push -f*)",
      "Edit(docs/*)",
    ]
  }
}
```

```

    "Write(docs/*)"
  ],
  "ask": [
    "Edit(.claude/settings.json)",
    "Write(.claude/settings.json)",
    "Edit(.claude/settings.local.json)",
    "Write(.claude/settings.local.json)"
  ]
},
"hooks": {
  "PostToolUse": [
    {
      "matcher": "Write|Edit",
      "hooks": [
        {
          "type": "command",
          "command": "jq -r '.tool_input.file_path // empty' | { read -r f; case \"$f\" in
            docs/*) npm run docs ;;
            *) ;;
          esac }",
          "timeout": 30,
          "statusMessage": "Regenerating docs/ from src/..."
        }
      ]
    }
  ]
}
}

```

A `PostToolUse` hook on the `Write|Edit` matcher: it reads the edited file's path from the hook's stdin JSON, and if the path is under `src/`, runs `npm run docs`. This is the automatic trigger — no manual step required after editing source.

Verification performed before trusting it: 1. Pipe-tested the raw command with synthetic stdin JSON: a `src/` path triggered regeneration, a non-`src/` path (`CLAUDE.md`) correctly did not. 2. Schema-validated with `jq -e '.hooks.PostToolUse[] | select(.matcher == "Write|Edit") | .hooks[] | select(.type == "command") | .command'` — exit 0. 3. Proved it fires for real: deleted `docs/`, made an actual `Edit` tool call to `src/main.jsx` (a throwaway `// hook-fire-test` comment, immediately reverted), and confirmed `docs/README.md` reappeared automatically with no manual `npm run docs` call. 4. Confirmed `npm run build` still passes afterward.

(f) Permissions notes

Guardrails were added to `.claude/settings.json` (team-wide, committed) alongside the hook. The user explicitly chose to leave the pre-existing `Bash(npm run *)` wildcard in `.claude/settings.local.json` untouched, and only add new deny/ask rules for operations this automation has no legitimate reason to perform:

Rule	Type	Reasoning
<code>Edit(docs/*) , Write(docs/*)</code>	deny	Enforces the <code>CLAUDE.md</code> rule "never hand-edit <code>docs/</code> " as a technical control, not just a written convention — forces all changes through the generator.
<code>Bash(rm -rf *)</code>	deny	The docs workflow never needs recursive force-delete; blocks it from ever being an unreviewed side effect.
<code>Bash(git push --force*) , Bash(git push -f*)</code>	deny	Nothing about doc generation should ever need a history rewrite.
<code>Edit / Write</code> on <code>.claude/settings.json</code> and <code>.claude/settings.local.json</code>	ask	Guards the guardrails — the automation's own config always requires explicit confirmation to change, so it can't silently expand its own permissions.

Honest finding from testing, not assumption: the `docs/*` deny rule was tested for real — by editing `docs/README.md` with the `Edit` tool (safe to test, since it's regenerable and not destructive). **The edit went through instead of being blocked.** Unlike the hook (confirmed to re-read live, see section e), permission rules appear to be snapshotted at session start rather than re-evaluated per call, so a mid-session change to `settings.json` isn't enforced until the session reloads/restarts. The test pollution was cleaned up immediately via `npm run docs`. The rule is syntactically valid (`jq -e` confirmed) and expected to take effect on the next session, but this document reports the gap as observed rather than assuming the rule works because the JSON was correct.

(g) Output from one full end-to-end automated run

No screenshot tool was available in this environment; the following is the actual terminal/tool output captured from one real, unscripted run — an `Edit` tool call was made to add a genuine JSDoc description to `src/components/Sidebar.jsx` (previously undocumented), with **no manual `npm run docs` call**, to prove the hook → generator → docs pipeline end to end.

1. Trigger — the edit that was made (via the `Edit` tool, not a manual doc regeneration):

```
+ /**
+  * Persistent left sidebar shown alongside the tab content in App.jsx:
+  * profile photo, quick info, social links, and a CV download button.
+  */
const SOCIALS = [
```

2. Automatic hook execution — the `PostToolUse` hook matched `src/components/Sidebar.jsx` and ran `npm run docs` on its own. Console output captured immediately after, with no intervening manual command:

```
> ajoydey00001-website@1.0.0 docs
> node scripts/generate-docs.js

Generated docs for 14 file(s) in docs/
```

3. Resulting `docs/src/components/Sidebar.jsx.md` (read immediately after, confirming the description flowed through automatically):

```
# Sidebar.jsx

`src/components/Sidebar.jsx`

Persistent left sidebar shown alongside the tab content in App.jsx: profile photo, quick info, social links, and a CV download button.

## Imports
_None_

## Imported by
- `src/App.jsx`

## Exports
### `Sidebar()`
No props.

## Local data constants
- `SOCIALS`
- `INFO_ITEMS`
```

4. Index table (`docs/README.md`) also updated automatically:

```
| [src/components/Sidebar.jsx](./src/components/Sidebar.jsx.md) | Sidebar | | Persistent left sidebar shown alongside the tab content in App.jsx: profile photo, quick info, social links, and a CV download button.
```

5. Final sanity check — confirming the automated regeneration didn't break the actual site build:

```
> ajoydey00001-website@1.0.0 build
> vite build

vite v5.4.21 building for production...
transforming...
✓ 39 modules transformed.
rendering chunks...
computing gzip size...
dist/index.html          1.62 kB | gzip:  0.68 kB
dist/assets/index-B_BjdXAU.css  24.29 kB | gzip:  4.87 kB
dist/assets/index-C6sU_xlf.js 169.69 kB | gzip: 52.98 kB
✓ built in 487ms
```

This is a permanent, real improvement (not a throwaway test) — `Sidebar.jsx` now has a genuine description where it previously had none, entirely as a side effect of normal editing.